

LAB 1 to 10

1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char text[100];
    int key, i;

    printf("Enter the plaintext: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter the key (1-25): ");
    scanf("%d", &key);

    printf("\nEncrypted Text: ");

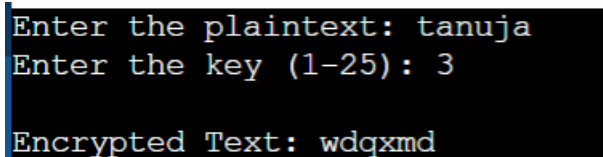
    for(i = 0; text[i] != '\0'; i++)
    {
        char ch = text[i];

        if(ch >= 'A' && ch <= 'Z')
        {
            ch = ((ch - 'A' + key) % 26) + 'A';
        }
        else if(ch >= 'a' && ch <= 'z')
        {
            ch = ((ch - 'a' + key) % 26) + 'a';
        }

        printf("%c", ch);
    }

    printf("\n");

    return 0;
}
```



```
Enter the plaintext: tanuja
Enter the key (1-25): 3

Encrypted Text: wdqxml
```

2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

```
#include <stdio.h>
```

```
#include <string.h>

int main()
{
    char plain[100], cipher[100];
    char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
    int i;

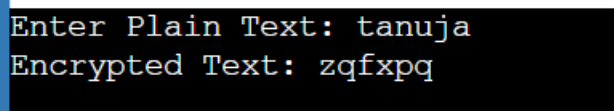
    printf("Enter Plain Text: ");
    scanf("%s", plain);

    for(i = 0; plain[i] != '\0'; i++)
    {
        if(plain[i] >= 'A' && plain[i] <= 'Z')
            cipher[i] = key[plain[i] - 'A'];
        else if(plain[i] >= 'a' && plain[i] <= 'z')
            cipher[i] = key[plain[i] - 'a'] + 32;
        else
            cipher[i] = plain[i];
    }

    cipher[i] = '\0';

    printf("Encrypted Text: %s\n", cipher);

    return 0;
}
```



```
Enter Plain Text: tanuja
Encrypted Text: zqfxpq
```

3. Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5];

void generateMatrix(char key[])
{
    int used[26] = {0};
    int i, k = 0;

    used['J' - 'A'] = 1;

    for(i = 0; key[i] != '\0'; i++)
    {
        char ch = toupper(key[i]);
```

```

        if(ch == 'J')
            ch = 'I';

        if(ch >= 'A' && ch <= 'Z' && !used[ch - 'A'])
        {
            matrix[k / 5][k % 5] = ch;
            used[ch - 'A'] = 1;
            k++;
        }
    }

    for(i = 0; i < 26; i++)
    {
        if(!used[i])
        {
            matrix[k / 5][k % 5] = i + 'A';
            k++;
        }
    }
}

void findPosition(char ch, int *row, int *col)
{
    int i, j;

    if(ch == 'J')
        ch = 'I';

    for(i = 0; i < 5; i++)
    {
        for(j = 0; j < 5; j++)
        {
            if(matrix[i][j] == ch)
            {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

int main()
{
    char key[50], text[100];
    int i, r1, c1, r2, c2;

    printf("Enter Key: ");
    scanf("%s", key);

    generateMatrix(key);

```

```

printf("\nPlayfair Matrix:\n");

for(i = 0; i < 5; i++)
{
    int j;
    for(j = 0; j < 5; j++)
        printf("%c ", matrix[i][j]);
    printf("\n");
}

printf("\nEnter Plain Text (Even letters): ");
scanf("%s", text);

printf("Encrypted Text: ");

for(i = 0; text[i] != '\0'; i += 2)
{
    char a = toupper(text[i]);
    char b = toupper(text[i + 1]);

    if(a == 'J') a = 'I';
    if(b == 'J') b = 'I';

    findPosition(a, &r1, &c1);
    findPosition(b, &r2, &c2);

    if(r1 == r2)
    {
        printf("%c%c",
            matrix[r1][(c1 + 1) % 5],
            matrix[r2][(c2 + 1) % 5]);
    }
    else if(c1 == c2)
    {
        printf("%c%c",
            matrix[(r1 + 1) % 5][c1],
            matrix[(r2 + 1) % 5][c2]);
    }
    else
    {
        printf("%c%c",
            matrix[r1][c2],
            matrix[r2][c1]);
    }
}

printf("\n");

return 0;
}

```

```

Enter Key: 3

Playfair Matrix:
A B C D E
F G H I K
L M N O P
Q R S T U
V W X Y Z

Enter Plain Text (Even letters):

```

4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

```

#include <stdio.h>
#include <string.h>

```

```

int main()
{
    char text[100], key[100], cipher[100];
    int i, j = 0;

    printf("Enter Plain Text: ");
    scanf("%s", text);

    printf("Enter Key: ");
    scanf("%s", key);

    int keyLen = strlen(key);

    for(i = 0; text[i] != '\0'; i++)
    {
        if(text[i] >= 'A' && text[i] <= 'Z')
        {
            cipher[i] = ((text[i] - 'A') + (key[j] % keyLen) - 'A')) % 26 + 'A';
            j++;
        }
        else if(text[i] >= 'a' && text[i] <= 'z')
        {
            cipher[i] = ((text[i] - 'a') + (key[j] % keyLen) - 'a')) % 26 + 'a';
            j++;
        }
        else
        {
            cipher[i] = text[i];
        }
    }

    cipher[i] = '\0';
}

```

```

printf("Encrypted Text: %s\n", cipher);

return 0;
}

```

```

Enter Plain Text: tanuja
Enter Key: 3
Encrypted Text: `MZaVM

```

5. Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$

A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.

a. Are there any limitations on the value of b ?

b. Determine which values of a are not allowed.

```

#include <stdio.h>
#include <string.h>

int main()
{
    char text[100], cipher[100];
    int a, b, i;

    printf("Enter Plain Text: ");
    scanf("%s", text);

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);

    for(i = 0; text[i] != '\0'; i++)
    {
        if(text[i] >= 'A' && text[i] <= 'Z')
        {
            cipher[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
        }
        else if(text[i] >= 'a' && text[i] <= 'z')
        {
            cipher[i] = ((a * (text[i] - 'a') + b) % 26) + 'a';
        }
        else
        {

```

```

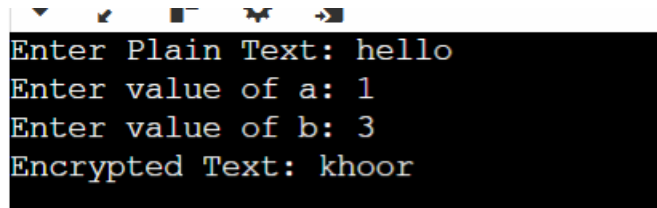
        cipher[i] = text[i];
    }
}

cipher[i] = '\0';

printf("Encrypted Text: %s\n", cipher);

return 0;
}

```



```

Enter Plain Text: hello
Enter value of a: 1
Enter value of b: 3
Encrypted Text: koor

```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is "B," and the second most frequent letter of the ciphertext is "U." Break this code.

```

#include <stdio.h>
#include <string.h>

```

```

int gcd(int a, int b)
{
    while(b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

```

```

int modInverse(int a)
{
    int i;
    for(i = 1; i < 26; i++)
    {
        if((a * i) % 26 == 1)
            return i;
    }
    return -1;
}

```

```

int main()
{
    char cipher[200];
    int a, b, i;

```

```

printf("Enter Cipher Text (Capital Letters): ");
scanf("%s", cipher);

printf("\nPossible Plain Texts:\n\n");

for(a = 1; a < 26; a++)
{
    if(gcd(a,26) == 1)
    {
        int inv = modInverse(a);

        for(b = 0; b < 26; b++)
        {
            printf("a=%d b=%d : ", a, b);

            for(i = 0; cipher[i] != '\0'; i++)
            {
                char ch = cipher[i];

                if(ch >= 'A' && ch <= 'Z')
                {
                    int p = (inv * ((ch-'A') - b + 26)) % 26;
                    printf("%c", p + 'A');
                }
                else
                {
                    printf("%c", ch);
                }
            }

            printf("\n");
        }
    }
}

return 0;
}

```



```
Enter Cipher Text (Capital Letters): TGWESBFKEWLD
```

```
Possible Plain Texts:
```

```
a=1 b=0 : TGWESBFKEWLD
a=1 b=1 : SFVDRAEJDVKE
a=1 b=2 : REUCQZDICUJB
a=1 b=3 : QDTBPYCHBTIA
a=1 b=4 : PCSAOXBGASHZ
a=1 b=5 : OBRZNWAFZRGY
a=1 b=6 : NAQYMVZEYQFX
a=1 b=7 : MZPXLUYDXPEW
a=1 b=8 : LYOWKTXCWODV
a=1 b=9 : KXNVJSWBVNCU
a=1 b=10 : JWMUIRVAUMBT
a=1 b=11 : IVLTHQUZTLAS
a=1 b=12 : HUKSGPTYSKZR
a=1 b=13 : GTJRFOSXRJYQ
a=1 b=14 : FSIQENRWQIXP
a=1 b=15 : ERHPDMQVPHWO
a=1 b=16 : DQGOCLPUOGVN
a=1 b=17 : CPFNBKOTNFUM
a=1 b=18 : BOEMAJNSMETL
```

7. Write a C program for the following ciphertext was generated using a simple substitution algorithm.

```
53†††305))6*;4826)4†.)4†);806*;48†8¶60))85;;]8*;;†*8†83
(88)5*†;46(;88*96*?;8)*†(;485);5*†2:*†(;4956*2(5*—4)8¶8*
;4069285;))6†8)4††;1(†9;48081;8:8†1;48†85;4)485†528806*81
(†9;48;(88;4(†?34;48)4†;161;:188;†?;
```

Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in

pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e.

2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.

3. Decipher the rest of the message by deducing additional words.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char cipher[500];
```

```
    char map[256];
```

```
    int i;
```

```
    for(i=0;i<256;i++)
```

```

    map[i]=0;

    printf("Enter Cipher Text:\n");
    fgets(cipher,sizeof(cipher),stdin);

    printf("\nEnter Mapping (Cipher -> Plain)\n");
    printf("Example: If 5 = A then enter A\n\n");

    printf("Mapping for 5 : ");
    scanf(" %c",&map['5']);

    printf("Mapping for 3 : ");
    scanf(" %c",&map['3']);

    printf("Mapping for 8 : ");
    scanf(" %c",&map['8']);

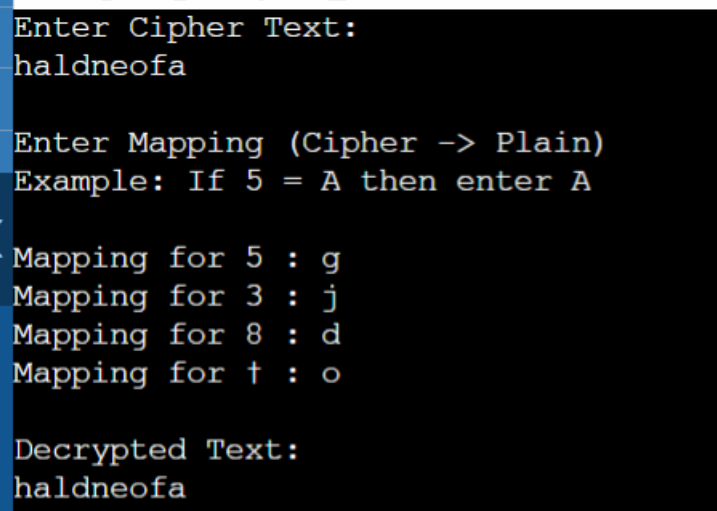
    printf("Mapping for † : ");
    scanf(" %c",&map[134]);

    printf("\nDecrypted Text:\n");

    for(i=0; cipher[i]!='\0'; i++)
    {
        if(map[(unsigned char)cipher[i]]!=0)
            printf("%c",map[(unsigned char)cipher[i]]);
        else
            printf("%c",cipher[i]);
    }

    return 0;
}

```



```

Enter Cipher Text:
haldneofa

Enter Mapping (Cipher -> Plain)
Example: If 5 = A then enter A

Mapping for 5 : g
Mapping for 3 : j
Mapping for 8 : d
Mapping for † : o

Decrypted Text:
haldneofa

```

8. Write a C program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from

which the cipher sequence can be generated.

For example, using the keyword CIPHER, write out the keyword followed by unused letters in normal order and match this against the plaintext letters:

```
plain: a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char plain[100], cipher[100];
    char key[] = "CIPHERABDFGJKLMNOQRSTUVWXYZ";
    int i;

    printf("Enter Plain Text (Capital Letters): ");
    scanf("%s", plain);

    for(i = 0; plain[i] != '\0'; i++)
    {
        if(plain[i] >= 'A' && plain[i] <= 'Z')
            cipher[i] = key[plain[i] - 'A'];
        else
            cipher[i] = plain[i];
    }

    cipher[i] = '\0';

    printf("\nCipher Alphabet : %s", key);
    printf("\nEncrypted Text : %s\n", cipher);

    return 0;
}
```

```
Enter Plain Text (Capital Letters): TANUJATARUN
Cipher Alphabet : CIPHERABDFGJKLMNOQRSTUVWXYZ
Encrypted Text : TCLUFCTCQUL
```

9. Write a C program for PT-109 American patrol boat, under the command of Lieutenant John F.

Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in

Playfair code:

```
KXJEY UREBE ZWEHE WRYTU HEYFS
KREHE GOYFI WTTTU OLKSY CAJPO
BOTEI ZONTX BYBNT GONEY CUZWR
GDSON SXBOU YWRHE BAAHY USEDQ
```

```
#include <stdio.h>
#include <string.h>
```

```

#include <ctype.h>

char matrix[5][5];

void generateMatrix(char key[])
{
    int used[26] = {0};
    int i, j, k = 0;

    used['J' - 'A'] = 1;

    for(i = 0; key[i] != '\0'; i++)
    {
        char ch = toupper(key[i]);

        if(ch == 'J')
            ch = 'I';

        if(ch >= 'A' && ch <= 'Z' && !used[ch - 'A'])
        {
            matrix[k / 5][k % 5] = ch;
            used[ch - 'A'] = 1;
            k++;
        }
    }

    for(i = 0; i < 26; i++)
    {
        if(!used[i])
        {
            matrix[k / 5][k % 5] = i + 'A';
            k++;
        }
    }
}

void findPos(char ch, int *r, int *c)
{
    if(ch == 'J')
        ch = 'I';

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 5; j++)
        {
            if(matrix[i][j] == ch)
            {
                *r = i;
                *c = j;
                return;
            }
        }
    }
}

```

```

    }
}

int main()
{
    char key[50], cipher[100];
    int r1, c1, r2, c2;

    printf("Enter Key: ");
    scanf("%s", key);

    generateMatrix(key);

    printf("Enter Cipher Text: ");
    scanf("%s", cipher);

    printf("\nDecrypted Text: ");

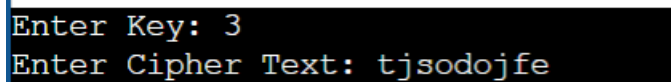
    for(int i = 0; cipher[i] != '\0'; i += 2)
    {
        findPos(cipher[i], &r1, &c1);
        findPos(cipher[i + 1], &r2, &c2);

        if(r1 == r2)
        {
            printf("%c%c",
                matrix[r1][(c1 + 4) % 5],
                matrix[r2][(c2 + 4) % 5]);
        }
        else if(c1 == c2)
        {
            printf("%c%c",
                matrix[(r1 + 4) % 5][c1],
                matrix[(r2 + 4) % 5][c2]);
        }
        else
        {
            printf("%c%c",
                matrix[r1][c2],
                matrix[r2][c1]);
        }
    }

    printf("\n");

    return 0;
}

```



```

Enter Key: 3
Enter Cipher Text: tjsodojfe

```

10. Write a C program for Playfair matrix:

M F H I/J K

U N O P Q

Z V W X Y

E L A R G

D S T B C

Encrypt this message: Must see you over Cadogan West. Coming at once.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char matrix[5][5] = {
    {'M','F','H','I','K'},
    {'U','N','O','P','Q'},
    {'Z','V','W','X','Y'},
    {'E','L','A','R','G'},
    {'D','S','T','B','C'}
};
```

```
void findPosition(char ch, int *row, int *col)
```

```
{
    if(ch == 'J')
        ch = 'I';

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 5; j++)
        {
            if(matrix[i][j] == ch)
            {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}
```

```
int main()
```

```
{
    char text[100];
    int r1, c1, r2, c2;

    printf("Playfair Matrix:\n");

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 5; j++)
            printf("%c ", matrix[i][j]);
        printf("\n");
    }
}
```

```
printf("\nEnter Plain Text (Even letters, No spaces): ");
scanf("%s", text);
```

```
printf("\nEncrypted Text: ");
```

```
for(int i = 0; text[i] != '\0'; i += 2)
{
    findPosition(text[i], &r1, &c1);
    findPosition(text[i + 1], &r2, &c2);
```

```
    if(r1 == r2)
    {
        printf("%c%c",
            matrix[r1][(c1 + 1) % 5],
            matrix[r2][(c2 + 1) % 5]);
```

```
    }
    else if(c1 == c2)
    {
        printf("%c%c",
            matrix[(r1 + 1) % 5][c1],
            matrix[(r2 + 1) % 5][c2]);
```

```
    }
    else
    {
        printf("%c%c",
            matrix[r1][c2],
            matrix[r2][c1]);
```

```
    }
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```

```
Playfair Matrix:
```

```
M F H I K
```

```
U N O P Q
```

```
Z V W X Y
```

```
E L A R G
```

```
D S T B C
```

```
Enter Plain Text (Even letters, No spaces): MUSTSEEYOUR^HOVER^
```

```
Encrypted Text: UZTBDLGZ
```